

# SOLUTION

## ***ECE 2036 Exam 1***

*Spring 2025*

**Instructions:** You have 1 hour, 15 minutes to complete this exam. The test is closed book and closed notes. No calculators, phones, or other electronic devices are allowed. Unless stated otherwise, justify your reasoning clearly to receive any partial credit. You must write your answer in the space provided on the exam paper itself. If space is needed for additional work, please use the backs of previous pages.

- I. **Compile or Not?:** If answer is no, indicate what compiler error(s) would occur! If answer is yes, write the output that would be produced, if any, when running the compiled program.

15 points

- a) Would this program compile with no compiler errors? (Yes or No) Yes

```
#include <iostream>
using namespace std;
```

```
int main()
{
    int diff, num1, _num2;

    num1 = 6.567;
    _num2 = 23;
    diff = num1 - _num2;
    cout << "Difference is " << diff << endl;
}
```

Difference is -17

- b) Would this program compile with no compiler errors? (Yes or No) Yes

```
#include <iostream>
using namespace std;
```

```
int main()
{
    int x=12, y=3;

    if (x >= 3)
        if (y > 3)
            cout << "x and y are > 3\n";
        else cout << "x is < 3\n";
}
```

x is < 3

- c) Would this program compile with no compiler errors? (Yes or No) Yes

```
#include <iostream>
using namespace std;
```

```
int main()
{
    int sum = 0;

    int i;
    for (i = 0; i < 10; i++)
    {
        sum += i;
    }

    cout << "Sum is " << sum << endl;
}
```

Sum is 10

## II. Fill in the blank:

10 points

Two types of numerical errors that occur in programs are called overflow errors and

precision errors. A clever programmer can reduce the amount of numerical error by carefully choosing the order of operations carried out by the program. Each object instantiated from a class has its own copy of the data members, while all objects share the same member functions.

A function defined outside of main() and not contained within any class is a global function.

A function prototype is when a programmer declares only the return type, name, and parameter types of a function but does not include the body of the function. A variable that is defined inside a

function and maintains its value across multiple function calls is a static variable. When an argument in a function call does not match the type of the corresponding parameter in the function

definition, C++ uses argument coercion to make the types agree and permit the function call to be made. Declaring a function to be inline advises the compiler to generate a copy of function's body code in *every* place where the function is called to avoid the overhead of function calls.

### III. Short answer:

10 points

- a) What is the value of `result` after the following C++ statements are executed?

9

```
int a = 2;  
int b = -7;  
double c = 6.4;  
double d = 2.0;  
int result = d * c + b / a;
```

Handwritten calculation for the expression `d * c + b / a`:  
 $2.0 \times 6.4 = 12.8$   
 $-7 / 2 = -3.5$   
 $12.8 - 3.5 = 9.3$   
The result is rounded down to 9.

- b) Suppose real numbers are represented with a base 10 floating point representation as follows. The mantissa is an 8-digit fraction with no leading zeros, the exponent is a 5-digit integer, and the mantissa and exponent both have one additional sign digit. For example, the number  $-123.456$  would be represented as:

$-.12345600 \times 10^{+00003}$

Write the value of the variables `result1` and `result2` in the above format after the following code segment is executed (Note: `float10` denotes our new base 10 floating point representation):

```
float10 f1 = 8001.0;  
float10 f2 = 2000.01;  
float10 f3 = 0.0001;  
float10 result1, result2;  
result1 = f1 + f2;  
result2 = result1 + f3;
```

Handwritten result for `result1`:  $+1.0001010 \times 10^{+00005}$

Handwritten result for `result2`:  $+1.0001010 \times 10^{+00005}$

#### IV. Converting between different kinds of loops

15 points

The following code when used with a Complex class as defined in Lab 2 and a global function findComplexRoot() to find a single root of a quadratic equation will allow a user to repeatedly enter a, b, and c from a quadratic equation  $ax^2 + bx + c$  and output the two (possibly complex) roots of the equation, terminating whenever the user enters 0 0 0 for a, b, and c.

```
int main()
{
    double a, b, c;
    Complex root1, root2;

    cin >> a >> b >> c;
    while ((a != 0) || (b != 0) || (c != 0)) {
        root1 = findComplexRoot(1, a, b, c);
        root2 = findComplexRoot(0, a, b, c);
        root1.outputAsString();
        root2.outputAsString();
        cin >> a >> b >> c;
    }
}
```

- a) Fill in the empty for statement in the below code to turn the code above, starting from the first "cin" statement, into a for loop. Do not add any additional statements, i.e. all additional code must go inside the empty for statement.

```
for (cin >> a >> b >> c; (a != 0) || (b != 0) || (c != 0); cin >> a >> b >> c) {
    root1 = findComplexRoot(1, a, b, c);
    root2 = findComplexRoot(0, a, b, c);
    root1.outputAsString();
    root2.outputAsString();
}
```

- b) Now, rewrite the same code as a "do, while" loop. Make sure that it has the exact same behavior as the original code. In particular, make sure that if a user enters "0 0 0" as their first inputs, the program terminates immediately without finding and outputting any roots.

```
cin >> a >> b >> c;
if ((a != 0) || (b != 0) || (c != 0))
{
    do
    {
        root1 = findComplexRoot(1, a, b, c);
        root2 = findComplexRoot(0, a, b, c);
        root1.outputAsString();
        root2.outputAsString();
        cin >> a >> b >> c;
    } while ((a != 0) || (b != 0) || (c != 0));
}
```

# V. Write member functions for a class:

20 points

The following is the header for a two-dimensional vector class. Write the code for the member functions of the class. The constructor should initialize the x and y variables of the vector to the parameter values provided. The add() and subtract() member functions work like the same member functions from Lab 2's Complex class. The dot product of two vectors (x1, y1) and (x2, y2) is  $x_1x_2 + y_1y_2$ .

```

class Vector2D {
public:
    Vector2D(double, double);
    double getX() const;
    double getY() const;
    void setX(double);
    void setY(double);
    Vector2D add(Vector2D) const;
    Vector2D subtract(Vector2D) const;
    double dotProduct(Vector2D) const;
private:
    double x;
    double y;
};
    
```

#include "Vector2D.h"

Vector2D::Vector2D(double xIn, yIn): x{xIn}, y{yIn} {}

double Vector2D::getX() const {  
    return x;  
}

double Vector2D::getY() const {  
    return y;  
}

void Vector2D::setX(double xIn) {  
    x = xIn;  
}

void Vector2D::setY(double yIn) {  
    y = yIn;  
}

Vector2D Vector2D::add(Vector2D v) const {  
    Vector2D vOut;  
    vOut.setX(x + v.getX());  
    vOut.setY(y + v.getY());  
    return vOut;  
}

Vector2D Vector2D::subtract(Vector2D v) const {  
    Vector2D vOut;  
    vOut.setX(x - v.getX());  
    vOut.setY(y - v.getY());  
    return vOut;  
}

double Vector2D::dotProduct(Vector2D v) const {

~~double dot~~  
    return (x \* v.getX() + y \* v.getY());  
}

## VI. Write a global function using pass by reference

20 points

Write a global function that calculates the mean and variance of a set of non-negative numbers. The function should prompt the user to enter the numbers one after the other and terminate the sequence with a negative number. You can assume there are no more than 1,000 numbers entered by the user and you can make use of a global array to store the numbers. The mean and variance should be returned to main() using reference parameters. The formula for the variance of n numbers is:

$$\sum_{i=1}^n (x_i - \mu)^2 / n$$

where  $x_i$  represents the  $i^{\text{th}}$  number and  $\mu$  represents the mean (note that you will first have to calculate the mean and then use that to calculate the variance, which will require two passes over the numbers). The global array and main() function to use with your global function are as follows:

```
double numbers[1000];
```

```
int main() {  
    double mean, variance;  
    findMeanAndVariance(mean, variance);  
    cout << "mean and variance are:\t" << mean << "\t" << variance << endl;  
}
```

Void findMeanAndVariance (double & mean, double & variance)

{

double sum = 0, varSum = 0, num;

int i = 0;

cin >> num;

while (num >= 0)

{

numbers[i] = num;

sum += num;

++i;

cin >> num;

}

mean = sum / i;

for (int j = 0; j < i; ++j)

{

varSum += (numbers[j] - mean) \* (numbers[j] - mean);

}

variance = varSum / i;

}

## VII. Program tracing:

10 points

Show the output of the following program.

```
#include <iostream>
using namespace std;

int times = 8;

void f2();

void f1(int & times)
{
    f2();
    times++;
    cout << times << endl;
}

void f2()
{
    static int times = 0;

    if (times < 3) {
        times++;
        cout << times << endl;
        f1(times);
    }
}

int main()
{
    cout << times << endl;
    f1(times);
    cout << times << endl;
}
```

8

1

2

3

4

5

6

9

9